

Project 2: Database Integration (CRUD)

Summary

The Journal is a backend project that lets users log a journal entry along with their mood for each day. Each user builds up their own history of entries over time, but the system won't let them log two entries for the same day by mistake. Full CRUD (create, read, update, delete) is implemented for both users and entries, and all the data is actually saved to a database, so it's still there even after the server restarts. The project uses FastAPI for the backend, SQLAlchemy as the ORM to talk to the database, and SQLite to store everything.

Requirement Analysis

- **Connecting to a database for persistence:** SQLAlchemy connects the API to a SQLite database, so instead of data disappearing when the server stops (like it would if it were just kept in memory), it's saved permanently to a file.
- **Designing a User schema:** The User model was kept simple, just an id, username, and email, with username and email both set as unique, forming the base for linking journal entries to whoever wrote them.
- **CRUD operations:** Endpoints were built to create users, and to create, read, update, and delete journal entries, plus an endpoint to list all entries belonging to a specific user.
- **Preventing duplicate entries:** The main duplicate rule is that a user can only have one entry per date, a unique constraint on the combination of user_id and date was added at the database level. If someone tries to add a second entry for a day they've already logged, the API catches it and sends back a proper 409 error instead of crashing. The same idea applies to usernames and emails, so no two users can sign up with the same one.
- **SQL basics and ORM concepts:** Since SQLAlchemy was used instead of raw SQL, the project shows how things like primary keys, foreign keys, and unique constraints translate into Python classes, and how the ORM turns that Python code into actual SQL behind the scenes.

FastAPI 0.1.0 OAS 3.1
/openapi.json

default

- GET /users List Users
- POST /users Create User
- POST /users/{user_id}/entries Create Entry
- GET /users/{user_id}/entries List Entries
- PUT /entries/{entry_id} Update Entry
- DELETE /entries/{entry_id} Delete Entry

Schemas

EntryCreate > Expand all object

FastAPI - Swagger UI

http://127.0.0.1:8000/docs/default/list_users_users_get

```
curl -X 'GET' \
  'http://127.0.0.1:8000/users' \
  -H 'accept: application/json'
```

Request URL

http://127.0.0.1:8000/users

Server response

Code	Details
200	Response body <pre>{ "id": 1, "username": "sara12", "email": "sara@example.com" }, { "id": 2, "username": "fara12", "email": "fara@example.com" }, { "id": 3, "username": "sam12", "email": "sam@example.com" } }</pre> Response headers <pre>content-length: 167 content-type: application/json date: Fri, 03 Jul 2026 12:00:35 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links

FastAPI - Swagger UI

http://127.0.0.1:8000/docs#/default/create_entry_users_user_id_entries_post

POST /users Create User

POST /users/{user_id}/entries Create Entry

Parameters

Cancel Reset

Name	Description
user_id * required	
integer	
(path)	

Request body * required

application/json

Edit Value Schema

```
{
  "date": "2026-07-03",
  "mood": "grateful",
  "text": "I am grateful"
}
```

FastAPI - Swagger UI

http://127.0.0.1:8000/docs#/default/create_entry_users_user_id_entries_post

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/users/1/entries' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "date": "2026-07-03",
    "mood": "grateful",
    "text": "I am grateful"
  }'
```

Request URL

http://127.0.0.1:8000/users/1/entries

Server response

Code	Details
409	Error: Conflict

Response body

```
{
  "detail": "Entry for this date already exists"
}
```

Response headers

```
content-length: 47
content-type: application/json
date: Fri, 03 Jul 2026 12:02:33 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	No links

Media type

FastAPI - Swagger UI

http://127.0.0.1:8000/docs#/default/list_entries_users_user_id__entries_get

GET /users/{user_id}/entries List Entries

Parameters

Name	Description
user_id * required integer (path)	3

Execute **Clear**

Responses

Curl

```
curl -X 'GET' \
  http://127.0.0.1:8000/users/3/entries' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/users/3/entries
```

Server response

Code	Details
200	

FastAPI - Swagger UI

http://127.0.0.1:8000/docs#/default/list_entries_users_user_id__entries_get

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/users/3/entries' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/users/3/entries
```

Server response

Code	Details
200	Response body <pre>{ "id": 3, "user_id": 3, "date": "2026-07-03", "mood": "fine", "text": "I am not angry anymore" }</pre> Response headers <pre>content-length: 88 content-type: application/json date: Fri, 03 Jul 2026 12:06:12 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links

FastAPI - Swagger UI

http://127.0.0.1:8000/docs#default/update_entry_entries_entry_id_put

PUT /entries/{entry_id} Update Entry

Parameters

Cancel Reset

Name	Description
entry_id * required	
integer (path)	3

Request body required

application/json

Edit Value Schema

```
{  "mood": "excited",  "text": "i am excited"}
```

Execute Clear

FastAPI - Swagger UI

http://127.0.0.1:8000/docs#default/update_entry_entries_entry_id_put

Responses

Curl

```
curl -X 'PUT' \  'http://127.0.0.1:8000/entries/3' \  -H 'accept: application/json' \  -H 'Content-Type: application/json' \  -d '{  "mood": "excited",  "text": "i am excited"  }'
```

Request URL

http://127.0.0.1:8000/entries/3

Server response

Code	Details
200	Response body <pre>{ "id": 3, "user_id": 3, "date": "2026-07-03", "mood": "excited", "text": "i am excited"}</pre> Download Response headers <pre>content-length: 79 content-type: application/json date: Fri, 03 Jul 2026 12:07:41 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links